

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»
КАФЕДРА ЕЛЕКТРОННИХ ОБЧИСЛЮВАЛЬНИХ МАШИН

ЗВІТ

до самостійної роботи

з дисципліни «Інструментальні засоби ВЕБ-технологій»

«Освітній портал StudyHub на основі WordPress та Next.js»

Варіант реалізації 2 — Decoupled CMS

Виконав:

СТУДЕНТ ГРУПИ КІСП-11

ПЯНИЛО ПАВЛО ВАДИМОВИЧ

Викладач: Ірина ПАСТЕРНАК

Львів — 2026

ЗМІСТ

1. Постановка завдання та вибір CMS

2. Архітектура і модель контенту

3. Реалізація порталу

4. Безпека, продуктивність і SEO

5. Розгортання та перевірка

6. Висновки

Список використаних джерел

Додаток А. Ілюстрації етапів розроблення

Структура звіту відповідає вимогам методичних вказівок до самостійної роботи. Ілюстрації відображають послідовні етапи розроблення, тому ранні знімки можуть відрізнятися від опублікованої версії.

1. ПОСТАНОВКА ЗАВДАННЯ ТА ВИБІР CMS

Мета роботи — розробити та опублікувати освітній портал StudyHub із відокремленою системою управління контентом і користувацьким інтерфейсом. Обрано другий варіант реалізації: WordPress як бекенд і Next.js як окремий фронтенд, що отримує матеріали через REST API.

Цільова аудиторія — студенти й початківці у веброзробці. Портал містить пояснення HTML, CSS, JavaScript та принципів роботи CMS. Відвідувач має знаходити матеріали за темою або текстовим запитом, відкривати статті та перемикаати українську й англійську версії.

WordPress обрано через готовий редактор, медіабібліотеку, категорії, ролі користувачів і відкритий REST API. Це дозволило зосередитися на інтеграції та моделі публікації. Next.js забезпечує серверне формування сторінок, маршрутизацію, кешування запитів і генерацію метаданих. Polylang забезпечує зв'язки між мовними версіями контенту.

Вимога	Реалізація
Керування матеріалами	Редактор WordPress, категорії та обкладинки
Навігація	Каталог, пошук, фільтри, пагінація по 3 матеріали
Мультимовність	Українська та англійська; маршрути / та /en
Погодження контенту	Contributor → Editor → опублікований матеріал
Публічний доступ	Фронтенд Vercel, CMS alwaysdata, HTTPS
SEO та адаптивність	Метадані, sitemap, robots; компонування для вузьких екранів

2. АРХІТЕКТУРА І МОДЕЛЬ КОНТЕНТУ

CMS і фронтенд розгорнуті окремо. Відвідувач відкриває сторінку Next.js, сервер отримує публічні дані WordPress, формує HTML і повертає його браузеру. WordPress зберігає статті, користувачів та таксономії в MariaDB. Обкладинки браузер завантажує з медіабібліотеки CMS.

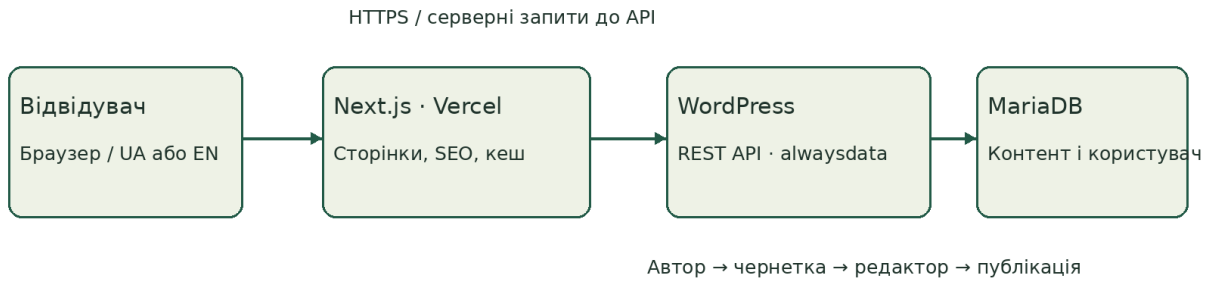


Рисунок 1 — Архітектура опублікованого рішення

Сутність	Основні поля	Зв'язки
Матеріал (post)	id, title, slug, content, excerpt, date, status	Автор; категорії; обкладинка; переклад
Користувач	id, name, role	Один користувач → багато матеріалів
Категорія	id, name, slug	Матеріали ↔ категорії: багато до багатьох
Медіафайл	id, source_url, alt_text, розміри	Матеріал → featured_media
Мовна версія	uk / en, посилання на переклад	Український матеріал ↔ англійський матеріал

Структура реалізована стандартними типами записів WordPress і таксономією category. Для роботи окремого фронтенду MU-плагін studyhub-language.php додає REST-поля studyhub_language та studyhub_translations. Останнє повертає slug лише опублікованих перекладів. Для запитів підтримується параметр lang, що обмежує результати відповідною мовою.

Роль	Призначення
Адміністратор — studyadmin	Налаштування CMS, плагінів і користувачів
Contributor — student_author	Створення чернеток і подання матеріалу на перевірку
Editor — student_editor	Редагування, перевірка та публікація матеріалів
Відвідувач	Читання публічного контенту через фронтенд

Редакційний процес: автор створює чернетку, подає її на перевірку, редактор перевіряє зміст і публікує. Матеріал «Як матеріал проходить редакційну перевірку» використано для демонстрації цього процесу. Неавторизований відвідувач не має прав публікувати записи або читати непублічні матеріали.

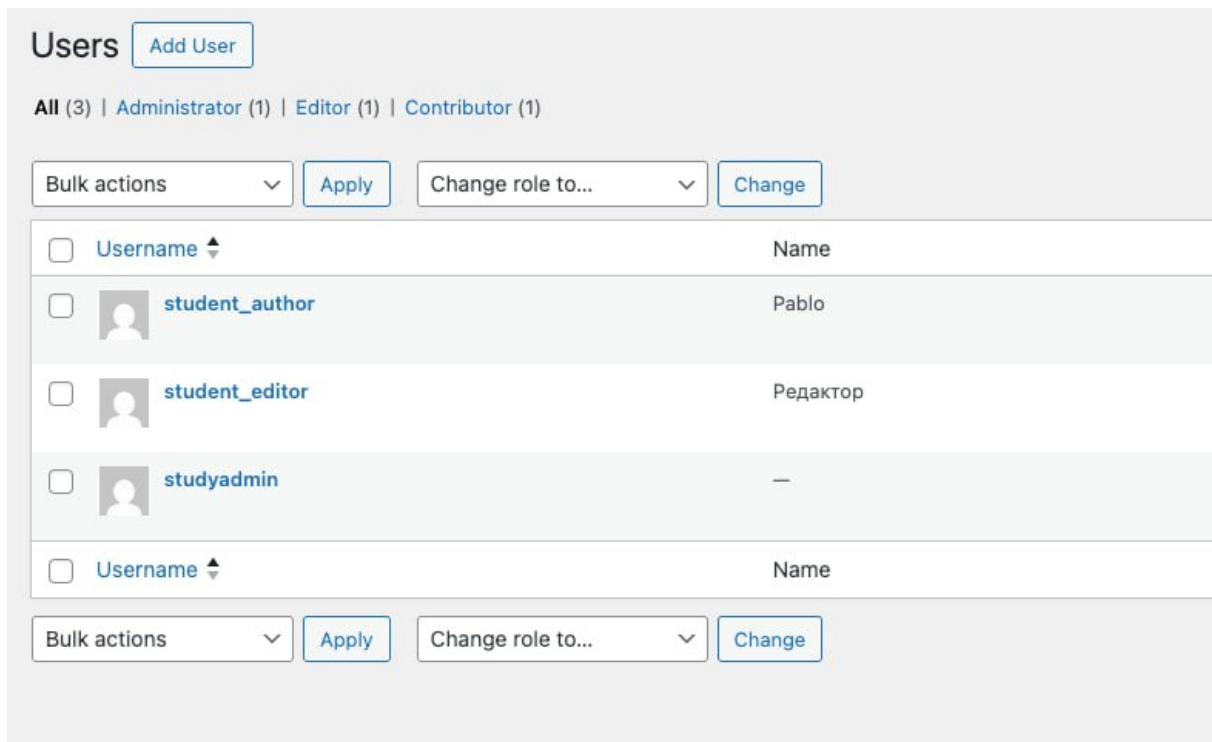


Рисунок 2 — Користувачі WordPress та розподіл ролей

3. РЕАЛІЗАЦІЯ ПОРТАЛУ

Локальне середовище побудоване за допомогою Docker Compose: окремі контейнери MariaDB, WordPress і WP-CLI. Фронтенд написано на React із TypeScript та Next.js App Router. Дані запитуються серверними компонентами; паролі CMS не передаються у клієнтський код.

Основні файли: `app/lib/wordpress.ts` — REST-запити та очищення HTML; `app/page.tsx` — каталог і пошук; `app/materialy/[slug]/page.tsx` — стаття; `app/lib/i18n.ts` — переклади інтерфейсу; `app/lib/seo.ts` — метадані; `wordpress/mu-plugins/studyhub-language.php` — мовне розширення API.

Приклад публічного запиту до CMS:

```
GET https://pavlo-pianylo.alwaysdata.net/
wp-json/wp/v2/posts?lang=uk&per_page=3&page=1
GET /wp-json/wp/v2/posts?lang=en&slug=html-en
```

Фрагмент серверного отримання каталогу (скорочено зі збереженням логіки):

```
url.searchParams.set('lang', language);
url.searchParams.set('per_page', String(PAGE_SIZE));
url.searchParams.set('page', String(page));
if (categoryId) url.searchParams.set('categories', String(categoryId));
if (search) url.searchParams.set('search', search);
const response = await fetch(url, {
  next: { revalidate: 60 },
```

```

    signal: AbortSignal.timeout(8000),
  });
  const posts = await response.json();

```

Пошук передає введений текст у параметр `search` WordPress і перевіряє також вміст статей. Тому пошук HTML може повернути статті про CSS і JavaScript, якщо вони містять це слово. Кількість сторінок визначається за заголовками `X-WP-Total` і `X-WP-TotalPages`. За порожнього результату показується зрозуміле повідомлення.

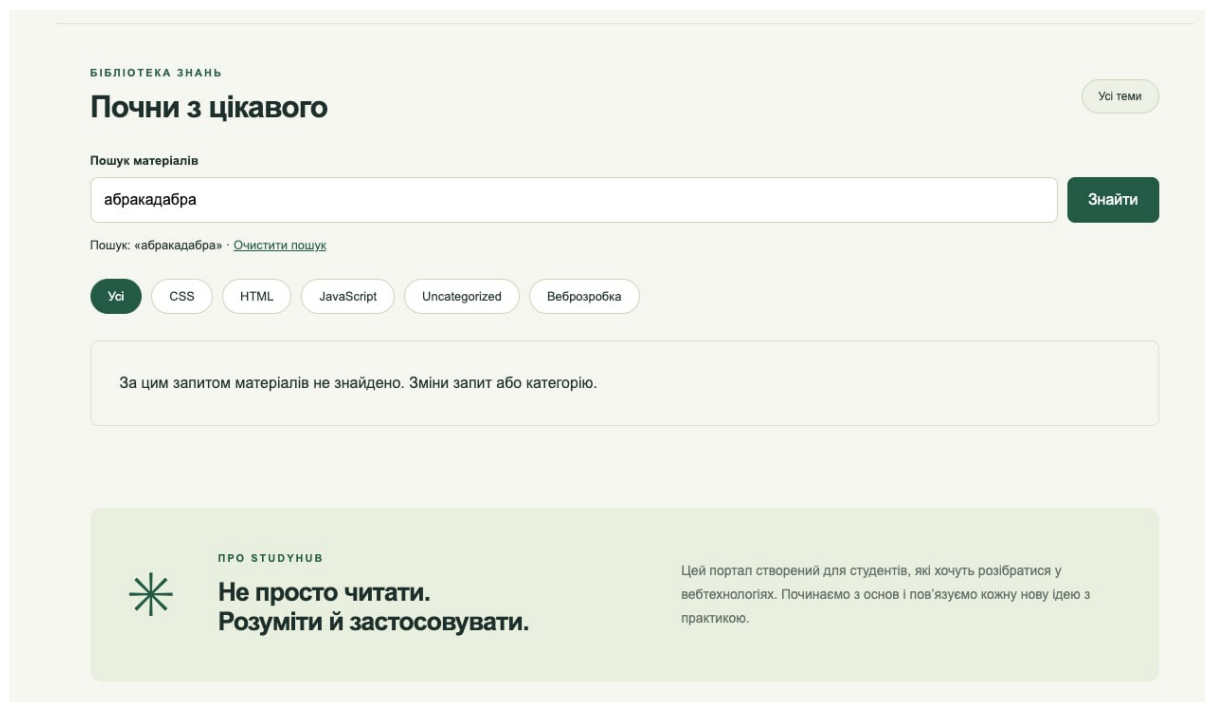


Рисунок 3 — Повідомлення про відсутність матеріалів за пошуковим запитом

Для статей налаштовані обкладинки в WordPress. Фронтенд отримує медіафайли через `_embed=author,wp:term,wp:featuredmedia`. Для зображень використовується альтернативний текст, а картки завантажують зображення відкладено. За відсутності обкладинки залишається графічне оформлення картки.

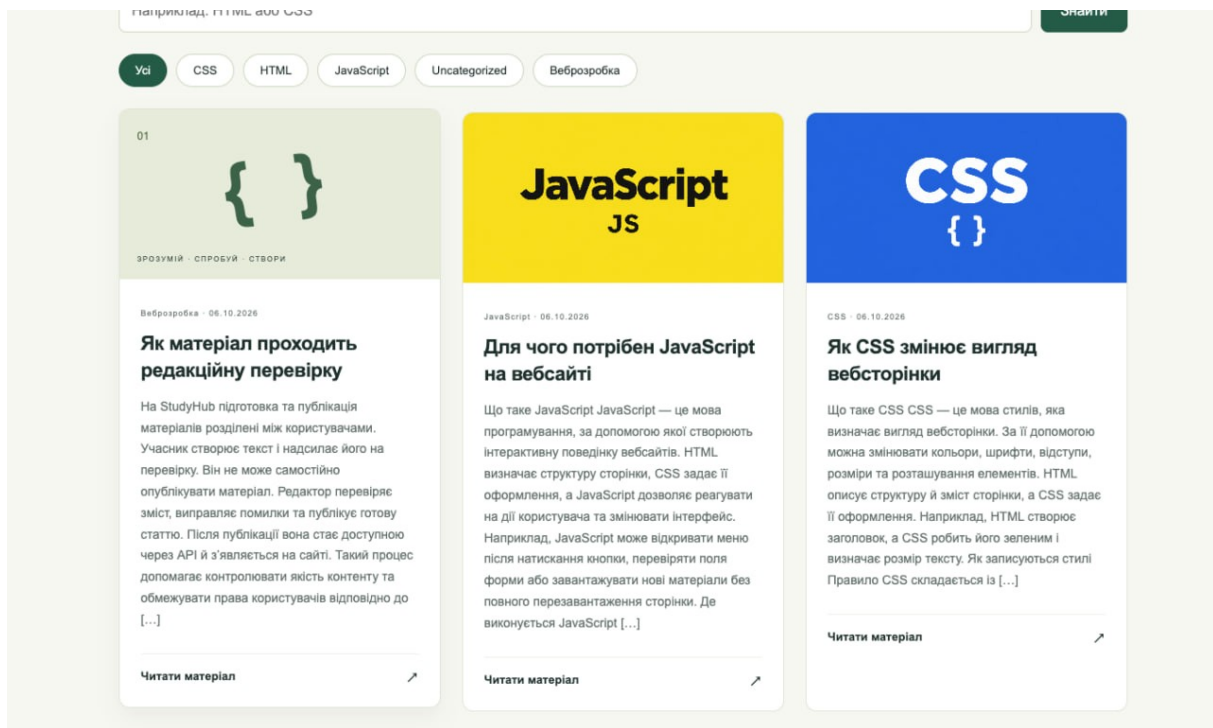


Рисунок 4 — Картки матеріалів із доданими обкладинками

Український контент відкривається на /, англійський — на /en. Каталог і категорії фільтруються за мовою. Посилання всередині статті відкриває її пов'язаний переклад; якщо його не опубліковано, показується повідомлення. Мова спільної шапки й підвалу визначається поточним маршрутом, атрибут `html lang` синхронізується під час переходів.

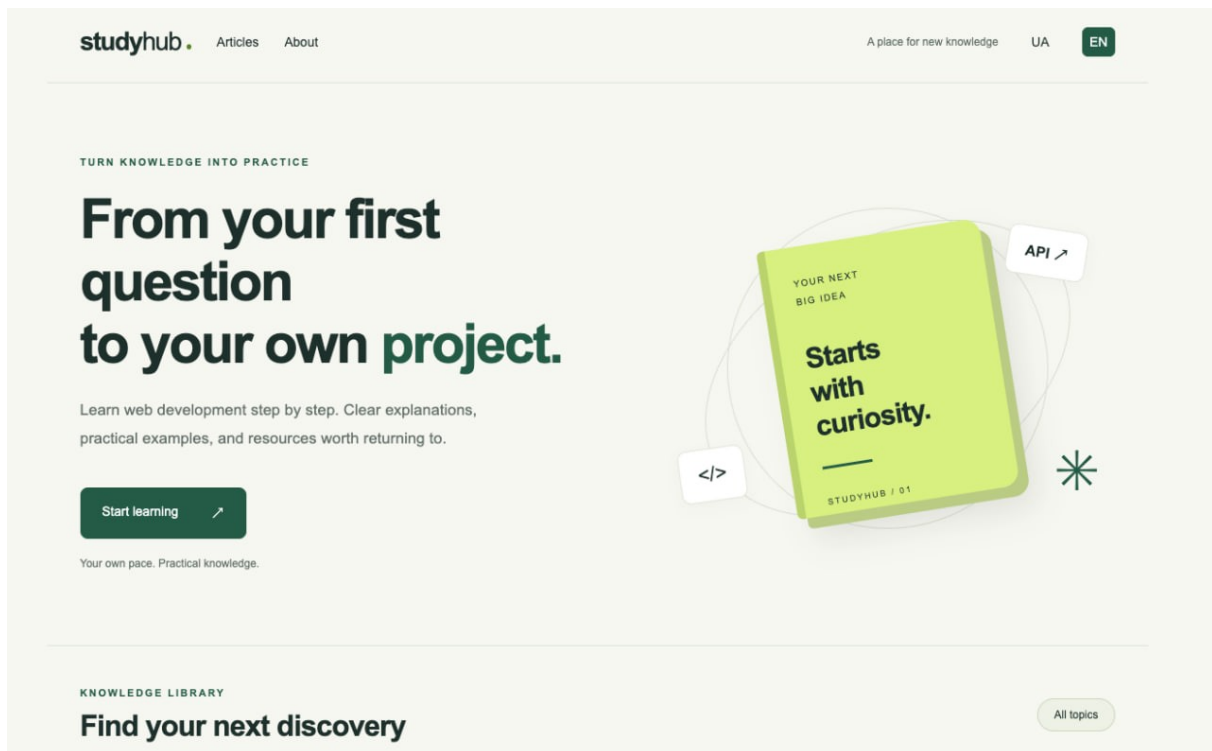


Рисунок 5 — Англійська версія інтерфейсу StudyHub

4. БЕЗПЕКА, ПРОДУКТИВНІСТЬ І SEO

Безпека реалізована на рівні доступу до CMS, серверних запитів і відображення контенту. Використані заходи зменшують ризики, описані OWASP; повний аудит на проникнення в межах роботи не проводився.

Ризик	Застосований захід
XSS	sanitize-html; дозволені теги й атрибути; заборона небезпечних схем URL
Порушення доступу	Ролі WordPress; REST API перевіряє права на запис і неопублічний контент
SQL Injection	Фронтенд не підключається до SQL; запити обробляються стандартним REST API WordPress
Витік секретів	Паролі залишаються в локальному .env та конфігурації сервера; .env виключено з Git
Перехоплення даних	Опубліковані адреси працюють через HTTPS
Залишені файли міграції	Після встановлення Duplicator видалив інсталятор та архів із кореня сайту

Очищення HTML виконується до вставлення тексту статті через dangerouslySetInnerHTML. Дозволено лише теги для текстового оформлення, списків, посилань і прикладів коду. Обробники подій і script не допускаються. Для заголовків та коротких описів використовується plainText.

```

sanitizeHtml(html, {
  allowedTags: ['p', 'h2', 'h3', 'h4', 'ul', 'ol', 'li',
    'strong', 'em', 'a', 'blockquote', 'pre', 'code', 'br'],
  allowedAttributes: { a: ['href', 'title'] },
  allowedSchemes: ['https', 'http', 'mailto'],
  allowProtocolRelative: false,
});

```

Продуктивність: Next.js повторно використовує результати REST-запитів із revalidate: 60. Каталог завантажує по три матеріали, категорії та sitemap обробляють усі сторінки API. Тайм-аут запиту становить 8 секунд. Такий кеш означає, що зміни CMS можуть з'являтися із затримкою; миттєве оновлення через webhook у поточній версії не реалізовано.

SEO: окремі title та description, canonical URL, hreflang для мовних версій, Open Graph і Twitter Card. Sitemap включає публічні матеріали обох мов; robots.txt містить його адресу. Пошукові й відфільтровані сторінки мають noindex, follow, а локальна версія закрита для індексації. Це технічна готовність до індексації, а не підтвердження появи сайту в Google.

Для українських slug враховано формат WordPress із процентним кодуванням: адреси формуються після нормалізації сегмента. Це запобігає повторному кодуванню знака % у sitemap та canonical URL.

5. РОЗГОРТАННЯ ТА ПЕРЕВІРКА

Публічна версія фронтенду: <https://web-independent-work.vercel.app/> .
 CMS: <https://pavlo-pianylo.alwaysdata.net/> . Вихідний код:
<https://github.com/pavlopianylomkisp2026/web-independent-work> .

WordPress перенесено з локального Docker на безкоштовний тариф alwaysdata. Створено MySQL-базу, налаштовано PHP-сайт і каталог www. Плагін Duplicator створив архів сайту та PHP-інсталятор; файли завантажено через SFTP. Під час встановлення введено параметри нової бази та замінено локальну адресу CMS публічною. Після входу адміністратора виконано очищення інсталяційних файлів.

Фронтенд імпортовано у Vercel із GitHub, обрано Next.js і кореневий каталог репозиторію. Налаштовано WORDPRESS_API_URL для публічного REST API та SITE_URL для адрес метаданих. Після зміни змінних середовища

виконано повторне розгортання. Автоматичні розгортання використовують гілку main.

WORDPRESS_API_URL=https://pavlo-piannylo.alwaysdata.net/wp-json/wp/v2
SITE_URL=https://web-independent-work.vercel.app

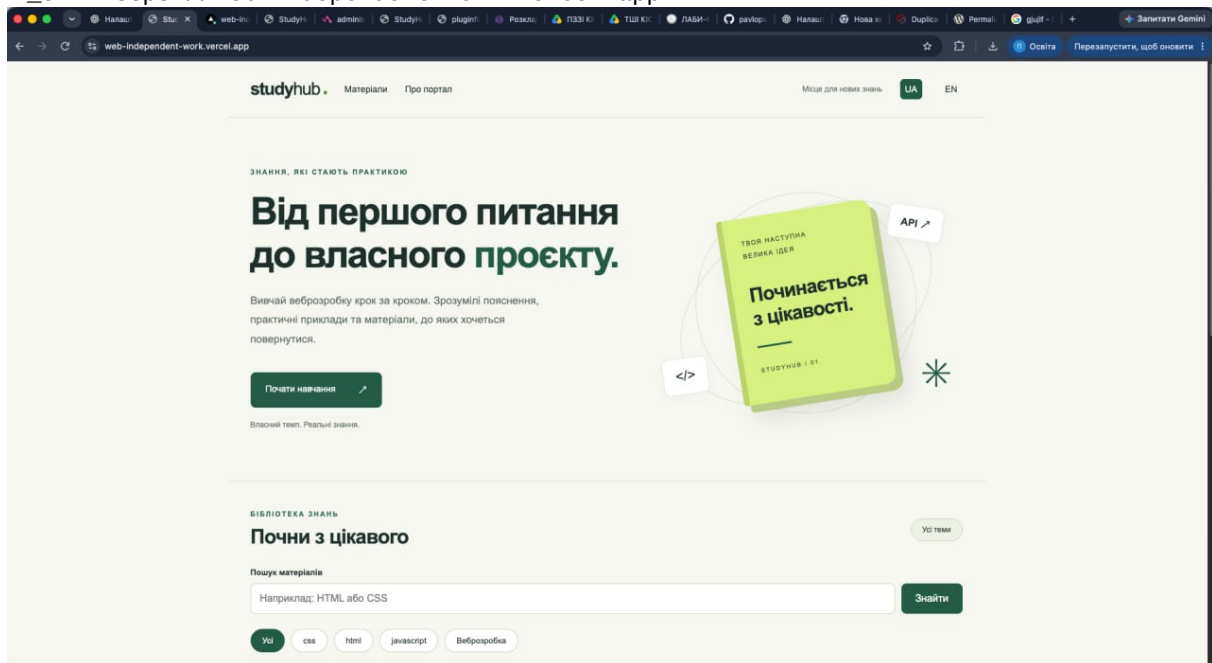


Рисунок 6 — Головна сторінка опублікованого фронтенду на Vercel

Перевірки виконувалися двома способами: автоматично в середовищі розроблення та вручну на локальному й опублікованому сайті. Скриншоти підтверджують відображення конкретних станів; вони не замінюють повного тестування безпеки або всіх мобільних екранів.

Перевірка	Результат / межі перевірки
Збірка та TypeScript	npm run build і npm run typecheck успішні
Базовий smoke	API, стаття, 404, категорії, пошук, пагінація та відмова в неавторизованому записі перевірені локально
Мовний smoke	Ізоляція мов і зв'язки перекладів перевірені локально на тестових матеріалах
SEO smoke	Sitemap, robots, canonical, alternates, Open Graph і PNG перевірені локально
Публічна версія	Користувач перевіряв кнопки; надані скріншоти головної, robots і sitemap
Мобільний вигляд	Надано скріншот головної сторінки на телефоні; повний набір пристроїв не перевірявся

6. ВИСНОВКИ

Розроблено освітній портал за архітектурою Decoupled CMS: управління

контентом у WordPress відокремлено від React-фронтенду на Next.js. Реалізовано каталог, сторінки статей, пошук, фільтрацію, пагінацію, обкладинки, ролі та українську й англійську версії. Обидві частини опубліковано, налаштовано кешування і технічні засоби SEO.

Основні труднощі: налаштування Docker на Mac; розрізнення паролів WordPress, бази даних і SSH; перенесення CMS; оновлення шапки під час перемикання мов; формування URL для кириличних slug. Для їх усунення використано покрокове налаштування, окремі серверні змінні, Duplicator, компоненти навігації на основі маршруту та нормалізацію адрес.

Перспективи розвитку: автоматичне скидання кешу після публікації, ширші мобільні й браузерні тести, регулярні резервні копії, моніторинг доступності, додаткові навчальні матеріали й особистий прогрес користувача. AI-функції залишено можливим розширенням; у поточний застосунок окремий AI API не інтегровано.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Пастернак І. І. *Методичні вказівки до самостійної роботи з дисципліни «Інструментальні засоби ВЕБ-технологій»*. Львів: НУ «Львівська політехніка», 2026. 5 с.
2. *Стилі кафедри ЕОМ. Наданий кафедральний файл стилів Word, редакція у назві файлу 20251004.*
3. *WordPress Developer Resources.* <https://developer.wordpress.org/>
4. *Next.js Documentation.* <https://nextjs.org/docs>
5. *Polylang Documentation.* <https://polylang.pro/doc/>
6. *Vercel Documentation.* <https://vercel.com/docs>
7. *alwaysdata Documentation.* <https://help.alwaysdata.com/>
8. *OWASP Top 10.* <https://owasp.org/Top10/>

ДОДАТОК А. ІЛЮСТРАЦІЇ ЕТАПІВ РОЗРОБЛЕННЯ

Нижче наведено додаткові скріншоти, зроблені під час розроблення. Службовий початковий запис Hello world! на ранніх ілюстраціях не є цільовим навчальним матеріалом.

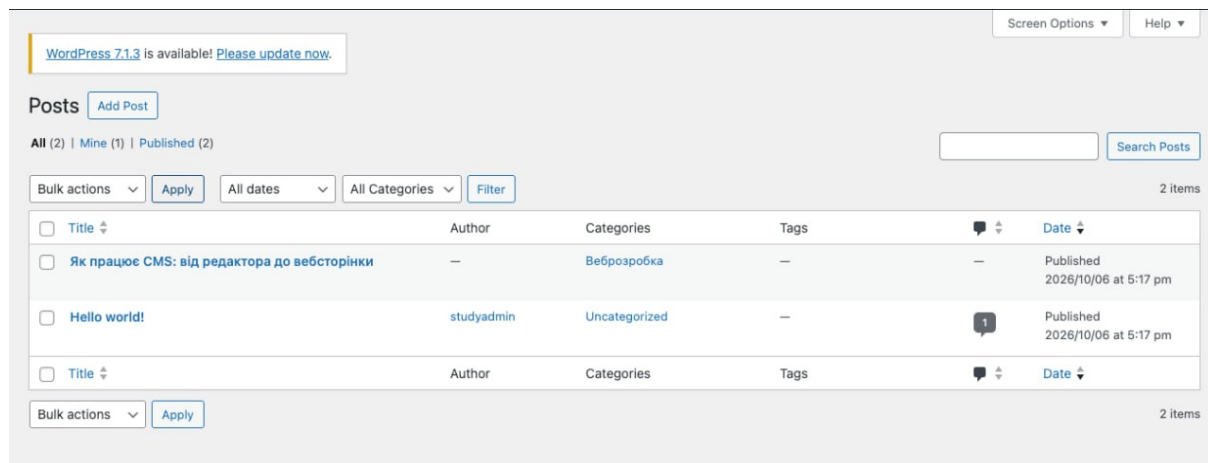


Рисунок 7 — Список записів у WordPress на початковому етапі

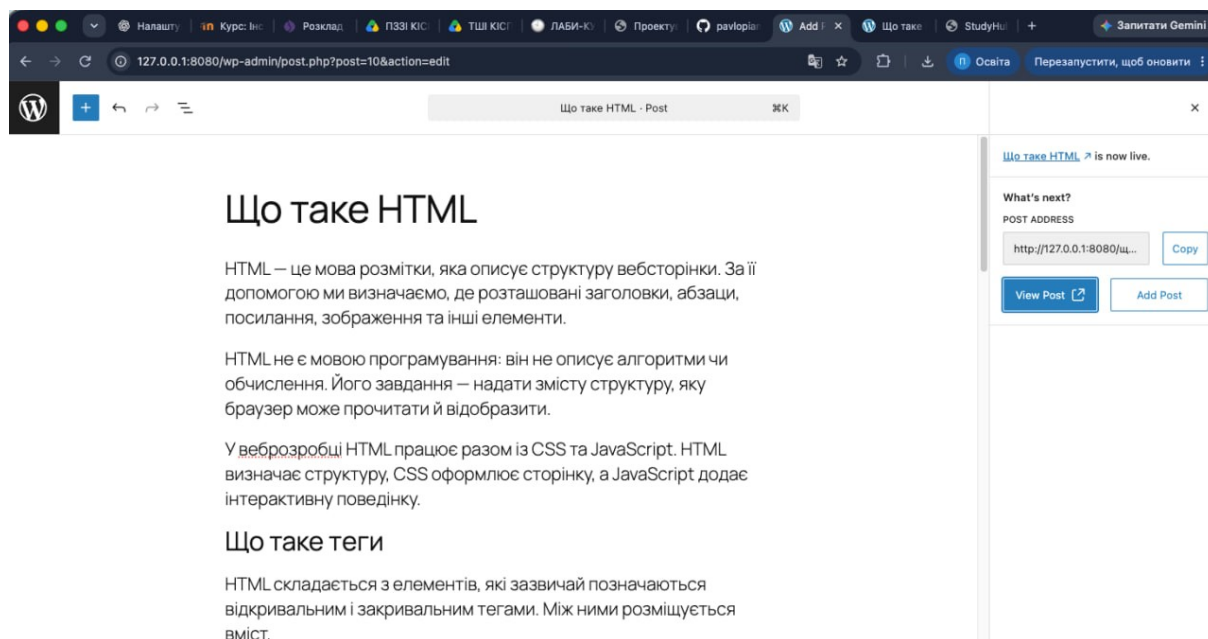


Рисунок 8 — Редагування навчального матеріалу «Що таке HTML»

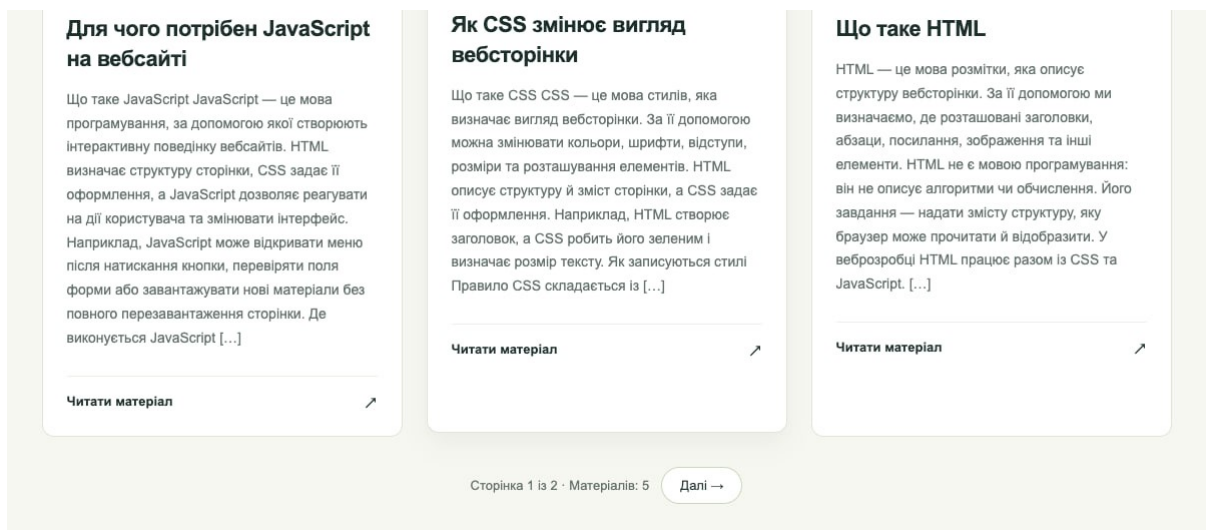


Рисунок 9 — Пагінація каталогу матеріалів

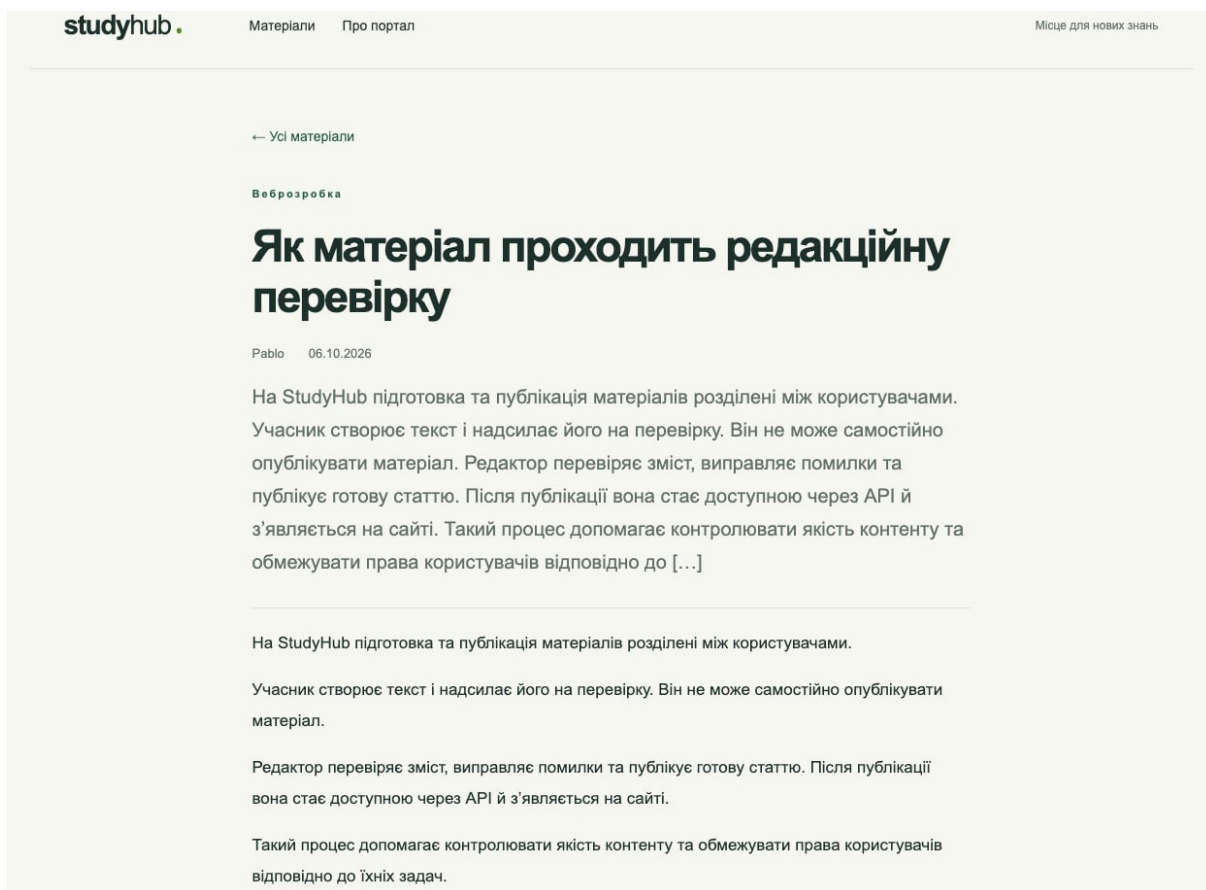


Рисунок 10 — Опублікований матеріал про редакційну перевірку